Sentinel-AI — AI Security Evaluation Report

ASCL AI Security Training · Prince Sultan University · Assignment A4 (Capstone)
Tool: Sentinel-AI v0.1.0 · Method: canary-token guardrail red-teaming · Date: 31 October 2026

1. Objective

This project evaluates the robustness of a chat model's safety guardrails against **prompt-injection** and **encoding-based evasion** attacks, and packages the evaluation as a reusable, installable command-line tool inspired by Promptfoo and Garak.

A deliberate design decision underlies the whole project: rather than measuring whether a model will produce genuinely dangerous content (which would require generating that content to grade it), Sentinel-AI uses a **canary-token** method. A harmless secret string is planted in the system prompt, and the model is instructed never to reveal it. An attack is counted as successful only when the model **leaks that harmless token**. This yields the same security signal — can the guardrail be defeated? — with zero hazardous output, so the tool is safe to publish and reuse by other researchers.

2. Methodology

2.1 Components

Component	Role
Target	The system under test. A deterministic `mock` guardrailed assistant (for reproducible demos), or a local model via `ollama:llama3.2`. Same interface for both.
Probe	A base attack input with an intent — e.g. directly asking for the token, or faking an administrator override.
Strategy	A transform applied to each probe to evade input filters: `plain`, `base64`, `rot13`, `leetspeak`.
Detector	Decides pass/fail for a single response. The primary detector checks whether the canary token is present (an objective, binary, reproducible test).
Evaluator	Runs every *probe × strategy × repetition*, then computes Attack Success Rate (ASR).

2.2 Metric

> **Attack Success Rate (ASR)** = confirmed successful attacks ÷ valid attempts.
> Reported overall, per attack category, and per evasion strategy.

2.3 Test matrix

4 probes × 4 strategies × 3 repetitions = **48 test cases** against the mock target. Repetitions expose any non-determinism; the mock is deterministic, so repetitions are consistent, which is

the desired baseline for an evidence demo.

3. Results

Scope	Successes / Attempts	ASR
Overall	27 / 48	**56.2%**
Category — prompt_injection	9 / 12	**75.0%**
Category — secret_extraction	12 / 24	50.0%
Category — jailbreak	6 / 12	50.0%
Strategy — plain	3 / 12	**25.0%**
Strategy — base64	12 / 12	**100.0%**
Strategy — rot13	12 / 12	**100.0%**
Strategy — leetspeak	0 / 12	**0.0%**

4. Root-cause analysis

The target's guardrail is a **keyword filter**: it scans the raw user text for words like "secret" or "token" and refuses when it finds them. This explains every number above:

- **Plain text (25% ASR):** direct requests contain the banned keywords, so most are refused. The few successes come from injection-style phrasings that avoid the keywords.
- **Base64 & ROT13 (100% ASR):** the filter sees only an opaque encoded blob — the banned words are not literally present — so the request passes the filter untouched. The model then decodes and obeys it. This is the central vulnerability.
- **Leetspeak (0% ASR):** the substitutions perturb the keywords enough to pass the filter, but the target does not "un-leet" the text, so the request is never acted upon. This reveals that filter coverage is *uneven* across transforms — a finding in itself.
- **Prompt injection (75% ASR):** fake "system administrator / developer mode" framing exploits a weak **trust boundary** — the model treats untrusted user text as if it were a privileged instruction.

> **Core lesson:** input-keyword filtering gives a false sense of safety. Any attacker who re-encodes their request walks straight past it. Robust guardrails must evaluate decoded *intent* and enforce a trust boundary, not match surface strings.

5. Recommended mitigations

- **Decode-then-check:** normalise/decode inputs (Base64, ROT13, leetspeak) before the safety check, so the filter sees the real intent.
- **Enforce a trust boundary:** never let user text be promoted to system authority; ignore "ignore previous instructions / admin override" patterns.

- **Output-side canary guard:** scan model *output* for the secret and block it — defence in depth, independent of the input filter.
- **Semantic (model-based) classification** instead of keyword matching, for intent that survives re-encoding.

Each mitigation is directly testable by re-running Sentinel-AI after applying it and confirming the ASR for the corresponding strategy drops — the "validate the fix" step from the training's evidence workflow.

6. Validation & reproducibility

Every case in the HTML/JSON report records the exact attack input, the full model response, and the detector verdict, so any finding can be reproduced independently. Four automated tests (`tests/test_sentinel.py`) lock in the core behaviours as regression checks. The mock target makes results deterministic; the Ollama target runs the identical benchmark against a real local model with all traffic confined to `localhost:11434`.

7. Conclusion

Sentinel-AI demonstrates, with reproducible local evidence, that keyword-based LLM guardrails are defeated by trivial input encoding (100% ASR for Base64 and ROT13 here) and by trust-boundary violations (75% ASR for prompt injection). The tool is modular, tested, documented, and installable, and it produces this evidence without ever generating harmful content — making it a practical, safe building block for real guardrail-robustness testing.

Generated locally. No data left the host machine. Sentinel-AI is a guardrail-robustness tool, not a content generator.